EECE 2140

Computing Fundamentals for Engineers

System Design Overview

Smart Parking Reservation Simulator

A C++ Web-Based Parking Management Project

Team Members: **Sitarama Challapalli**
Adrian Zielinski
John Nevin

Instructor: Dr. Fatema Nafa

Semester: Spring 2026

Repository: https://github.com/saicharanchallapalli10-crypto/Smart-Parking-Reservation-Simulator-.git

Contents

1 Project Overview **2**
1.1 Motivation . . . 2
1.2 Project Goals . . . 2

2 Basic Functionalities **2**
2.1 Functionality Summary Table . . . 3
2.2 Detailed Functionality Descriptions . . . 3

3 Object-Oriented Programming (OOP) Design **4**
3.1 Class Relationship Overview . . . 4
3.2 OOP Design Table . . . 5
3.3 Key OOP Requirements . . . 6

4 System Overview; Input, Process, Output **7**
4.1 High-Level Description . . . 7
4.2 IPO Diagram . . . 7
4.3 Detailed Step Descriptions . . . 7
4.3.1 Input . . . 7
4.3.2 Process . . . 7
4.3.3 Output . . . 8

5 Pseudocode **8**

6 C++ Implementation Highlights **9**

7 Challenges, Solutions & Lessons Learned **10**
7.1 Challenges & Solutions . . . 11
7.2 Lessons Learned . . . 11

8 Limitations & Future Work **12**

9 GitHub Repository Structure **13**
9.1 How to Compile and Run . . . 13

10 Tools and Technology Used **14**

References **14**

A Example Program Output **14**
A.1 Successful Reservation . . . 15
A.2 Spot Release . . . 15
A.3 Status Table Example . . . 16
A.4 Error Handling . . . 16

1 Project Overview

Project Description

The **Smart Parking Reservation Simulator** is a `C++` web-based application that simulates a 100-spot parking lot. Users can reserve and release parking spots through a browser interface, and the system charges a dynamic fee that adjusts based on how full the lot is and how many hours are requested. The `C++` backend handles all reservation logic, fee calculation, and spot management. The HTML and JavaScript frontend provides the visual interface, and the two communicate over HTTP using the `httplib` library.

1.1 Motivation

Parking management is a common challenge in cities, campuses, and commercial areas. Drivers waste time looking for open spots, and flat-rate pricing does not reflect real demand. Professional platforms such as ParkMobile and SpotHero solve this with demand-based pricing that adjusts in real time.

This project applies that concept at an academic level. Instead of building a production system, we built a simulator that models the core mechanics of parking reservation; tracking availability, registering users, and calculating fees based on occupancy. It gave us a practical setting to apply object-oriented programming beyond textbook exercises.

1.2 Project Goals

- Design a system using separate classes that each handle one responsibility; user information, spot management, and pricing logic.
- Implement dynamic pricing that adjusts based on lot occupancy and parking duration rather than using a flat rate.
- Connect the `C++` backend to a web-based frontend so users can interact with the system through a browser instead of a console.
- Collaborate as a team using GitHub for version control and task division.
- Write clean, well-structured, and commented code that is easy to follow and maintain.

2 Basic Functionalities

This section covers the core features of the simulator; what they take in, what they produce, and how they contribute to the system.

2.1 Functionality Summary Table

#	Functionality	Description	Input	Output
F1	**User Registration**	Creates a temporary `User` object storing the driver's name and car model, then passes that data into the reservation process.	Name, car model from the web form	`User` object with getter values passed to `ParkingSpot`
F2	**Spot Reservation**	Validates the requested spot, checks availability, calculates the dynamic fee, and marks the spot as taken.	Spot number, hours, user details	Confirmation message, calculated fee, updated lot counts
F3	**Dynamic Pricing**	Calculates the parking fee using the formula $(\text{baseRate} + \text{takenSpots} \times 0.30) \times \text{hours}$. The price rises as more spots are occupied.	Base rate, number of occupied spots, hours	Final fee as a dollar amount
F4	**Spot Release & Status**	Releases an occupied spot by clearing its data back to default. Also provides a full status view of all 100 spots.	Spot number (for release); full array (for status)	Goodbye message with updated counts; or a complete HTML status table

2.2 Detailed Functionality Descriptions

F1; User Registration

Description: When a user submits the reservation form, main.cpp creates a temporary User object from the submitted name and car model. The getter methods getName() and getCarModel() pass these values into ParkingSpot::reserveSpot(); once the handoff is complete the object goes out of scope.

Input: Name and car model (string) from the HTTP request.
Output: Getter values passed into ParkingSpot as raw strings.
Contribution: Provides user identity data to the reservation process.

F2; Spot Reservation

Description: The /reserve route handler extracts the spot number and hours from the request and validates them; the spot must be between 1 and 100 and hours must be positive. If the spot is free, it counts the currently taken spots, calculates the fee through a PricingEngine, creates a User, and calls reserveSpot() to store the data. A JSON response containing the fee and updated lot counts is then sent back to the browser.

Input: Spot number (int), hours (int), name and car model (string).
Output: Response with success or error message; fee; and taken/available counts.
Contribution: Core transaction of the system; this is where a spot changes hands.

$ F3; Dynamic Pricing

Description: PricingEngine holds a baseRate of $5.00 and exposes a single method; calculateFee(int takenSpots, int hours). The formula is fee = (baseRate + takenSpots × 0.30) × hours. It receives the occupancy count from main.cpp each time and does not track occupancy itself.

Input: Number of taken spots (int); hours (int).
Output: Calculated fee (double).
Contribution: Provides demand-based pricing that makes the simulation realistic.

F4; Spot Release & Status

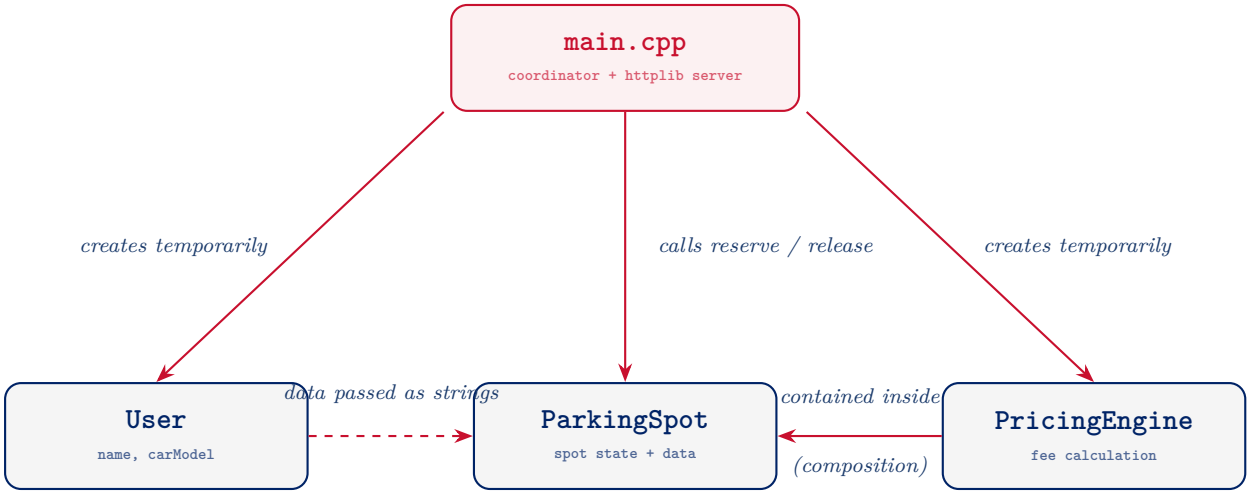
Description: The /release route validates the spot number and confirms that the spot is occupied; it then retrieves the occupant's name for a goodbye message and calls releaseSpot() to reset the spot. The /status route calls buildStatusPage(), which loops through all 100 spots and returns an HTML table showing their current state.

Input: Spot number (int) for release; no input for status.
Output: Goodbye message for release; full HTML page for status.
Contribution: Completes the reservation lifecycle and provides lot visibility.

3 Object-Oriented Programming (OOP) Design

3.1 Class Relationship Overview

The system uses **three classes**, each responsible for one part of the simulation. `User` handles identity, `ParkingSpot` handles state, and `PricingEngine` handles cost logic. The file `main.cpp` ties all three together and runs the web server.



3.2 OOP Design Table

Class	Attribute / Method	Access	Data Type	Description
`User`	`name`	private	`string`	User's name
	`carModel`	private	`string`	User's car model
	`User(string, string)`	public	constructor	Sets name and car model
	`getName()`	public	`string`	Returns user's name
	`getCarModel()`	public	`string`	Returns car model
	`registerUser()`	public	`void`	Prints registration confirmation
	`getUserInfo()`	public	`void`	Prints user information
`ParkingSpot`	`spotNumber`	private	`int`	Spot ID (1 to 100)
	`isAvailable`	private	`bool`	Whether spot is free
	`assignedTo`	private	`string`	Occupant's name
	`carModel`	private	`string`	Occupant's car model
	`reservedHours`	private	`int`	Hours reserved
	`pricing`	private	`PricingEngin`	Fee calculator (composition)
	`ParkingSpot(int)`	public	constructor	Sets spot number
	`getSpotNumber()`	public	`int`	Returns spot number

Class	Attribute / Method	Access	Data Type	Description
	`getIsAvailable()`	public	`bool`	Returns availability
	`getAssignedTo()`	public	`string`	Returns occupant name
	`getCarModel()`	public	`string`	Returns car model
	`getReservedHours()`	public	`int`	Returns reserved hours
	`reserveSpot(str, str, int, int)`	public	`void`	Reserves spot with user data
	`releaseSpot()`	public	`void`	Clears spot to available
	`checkSpot()`	public	`void`	Prints spot status to console
	`getStatusInfo()`	public	`string`	Returns text status summary
`PricingEngine`	`baseRate`	private	`double`	Base fee ($5.00)
	`PricingEngine()`	public	constructor	Sets default base rate
	`getBaseRate()`	public	`double`	Returns base rate
	`calculateFee(int, int)`	public	`double`	Computes fee from occupancy and hours

3.3 Key OOP Requirements

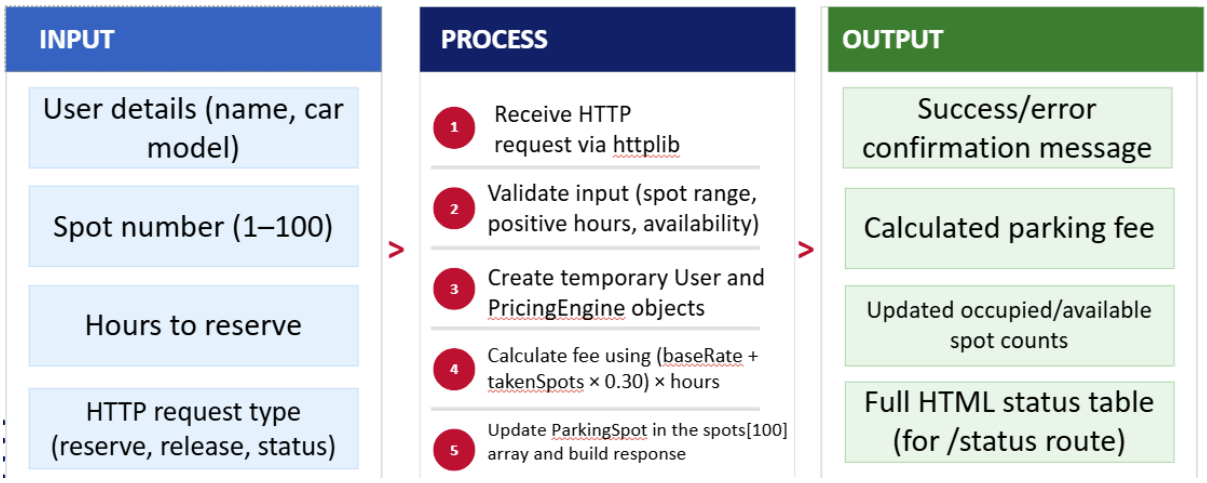
- **3 Classes:** `User`, `ParkingSpot`, `PricingEngine`; each with a distinct, single responsibility.
- **Private Attributes:** All data members are `private`; access is only through public getters and controlled methods like `reserveSpot()` and `releaseSpot()`.
- **Encapsulation:** Spot state, pricing logic, and user data are hidden from outside code. Interaction happens only through the public interface.
- **Class Composition:** Each `ParkingSpot` contains a `PricingEngine` as a private member, allowing it to calculate fees without depending on external objects.
- **Coordinator Pattern:** `main.cpp` is the single point of coordination; it creates objects, pulls data through getters, and passes values between classes. No class other than `main.cpp` knows the web server exists.

4 System Overview; Input, Process, Output

4.1 High-Level Description

The application follows a **client-server architecture**. The HTML and JavaScript frontend collects user input and sends it to the `C++` backend via HTTP requests. The backend processes the request through its classes, updates the in-memory parking lot array, and sends back a response that the frontend uses to update the page without reloading.

4.2 IPO Diagram



Data flow: `Frontend fetch()` → `main.cpp route handler` → `User / PricingEngine / ParkingSpot` → `spots[100] array` → `Response` → `Browser update`

4.3 Detailed Step Descriptions

4.3.1 Input

→ **User details:** Name and car model entered through the reservation form.

→ **Spot number:** An integer between 1 and 100 identifying the desired parking spot.

→ **Hours:** A positive integer for how long the user wants to reserve the spot.

→ **HTTP request type:** The route determines the action; `POST /reserve`, `POST /release`, `GET /summary`, or `GET /status`.

4.3.2 Process

When a request arrives at the `C++` backend:

Step 1. **Receive request.** The `httplib` server catches the HTTP request and routes it to the appropriate handler in `main.cpp`.

Step 2. **Validate input.** The handler checks that the spot number is in range (1 to 100), hours are positive, and availability matches the requested action.

Step 3. Create objects. A temporary `User` is created to hold identity data. A `PricingEngine` is created to calculate the fee.

Step 4. Calculate fee. The `PricingEngine` runs its formula using the current count of taken spots and the requested hours.

Step 5. Update and respond. The handler calls `reserveSpot()` or `releaseSpot()` on the appropriate `ParkingSpot`, then builds and sends a response back to the browser.

4.3.3 Output

→ **Confirmation message:** Success or error text displayed on the page without reloading.

→ **Calculated fee:** The dollar amount computed by the pricing formula.

→ **Updated counts:** Current number of occupied and available spots, refreshed after every action.

→ **Status table:** A full HTML table showing all 100 spots with their current state.

5 Pseudocode

Main Server Loop

```
PROGRAM ParkingServer
  INITIALIZE spots[100]
  START server

  ON RESERVE REQUEST:
    READ name, carModel, spotNumber, hours
    CALL reserveSpot(name, carModel, spotNumber, hours)

  ON RELEASE REQUEST:
    READ spotNumber
    CALL releaseSpot(spotNumber)

  ON STATUS REQUEST:
    LOOP through all 100 spots
    BUILD status table
    SEND HTML to browser
END PROGRAM
```

Function: reserveSpot

```
FUNCTION reserveSpot(name, carModel, spotNumber, hours):
  IF spot is available:
    COUNT taken spots
    CALCULATE fee using (baseRate + takenSpots x 0.30) x hours
    STORE name, carModel, hours into spot
    MARK spot as taken
    SEND success + fee to browser
```

```
  ELSE:
    SEND error to browser
  END IF
END FUNCTION
```

Function: releaseSpot

```
FUNCTION releaseSpot(spotNumber):
  IF spot is taken:
    CLEAR name, carModel, hours from spot
    MARK spot as available
    SEND success to browser
  ELSE:
    SEND error to browser
  END IF
END FUNCTION
```

6 C++ Implementation Highlights

/reserve Route Handler; Core Reservation Logic

```
svr.Post("/reserve", [&](const httplib::Request& req,
                         httplib::Response& res) {
    string name     = req.get_param_value("name");
    string carModel = req.get_param_value("carModel");
    int choice      = stoi(req.get_param_value("spotNumber"));
    int hours       = stoi(req.get_param_value("hours"));

    // Validate input
    if (choice < 1 || choice > 100 || hours <= 0) {
        res.set_content("{\"error\": \"Invalid input\"}",
                        "application/json");
        return;
    }

    // Check availability
    if (!spots[choice - 1].getIsAvailable()) {
        res.set_content("{\"error\": \"Spot taken\"}",
                        "application/json");
        return;
    }

    // Count demand and calculate fee
    int takenSpots = countTakenSpots();
    PricingEngine pe;
    double fee = pe.calculateFee(takenSpots, hours);

    // Create user and reserve
    User user(name, carModel);
    spots[choice - 1].reserveSpot(
```

```
            user.getName(), user.getCarModel(),
            takenSpots, hours);

        // Build and send response
        string json = buildJsonSummary(fee, name, choice);
        res.set_content(json, "application/json");
    });
```

Key Design Decisions

D1. In-memory array over database: All 100 spots live in a global `ParkingSpot spots[100]` array. This keeps the project focused on OOP without database complexity. The tradeoff is that all data resets when the server stops.

D2. Composition over inheritance: Each `ParkingSpot` contains its own `PricingEngine` member, keeping pricing self-contained within each spot.

D3. Temporary object pattern: `User` and `PricingEngine` objects are created on the stack during request handling and destroyed after use. This avoids maintaining a separate user registry.

D4. Frontend generated via AI prompting: The HTML, CSS, and JavaScript were generated through iterative AI prompting. This let the team focus on the `C++` backend and class design while still delivering a functional web interface.

7 Challenges, Solutions & Lessons Learned

7.1 Challenges & Solutions

#	Challenge	Solution
1	Understanding how `httplib` works took time; none of us had used a `C++` HTTP library before and the request/response model was unfamiliar.	Read through the `httplib` documentation and tested small route handlers individually before building out the full server.
2	Deciding what data belongs inside each class versus what should be handled in `main.cpp` was not obvious at first.	Applied the single-responsibility principle; if data relates to a spot, it goes in `ParkingSpot`. If it relates to pricing, it goes in `PricingEngine`. Everything else stays in `main.cpp`.
3	The `ParkingSpot` constructor needed to initialize both its own members and its internal `PricingEngine`, which introduced us to the concept of member initialization order.	Learned that member objects are constructed automatically when the parent object is created, and that default constructors handle the setup if no arguments are needed.
4	Testing the full system end-to-end was harder than testing classes individually; issues only appeared when the frontend and backend were running together.	Tested each HTTP route separately using the browser before combining them. Checked the browser developer console for errors after every change.

7.2 Lessons Learned

- **Private attributes catch mistakes early:** Keeping everything private forced all data to flow through controlled methods. This prevented accidental overwrites that would have been hard to trace.
- **Object lifetime is not just a concept:** Watching temporary `User` and `PricingEngine` objects get created and destroyed within a single function made scope and lifetime feel concrete.
- **Integration is its own challenge:** Writing the individual classes was the straightforward part. The real difficulty was getting `main.cpp` to call everything in the right order and pass data between classes correctly.
- **The browser console is your best friend:** Most frontend issues only became visible

by inspecting the network tab and console output. We would not have caught several response formatting errors without it.

8 Limitations & Future Work

Current Limitation	Proposed Future Extension
The server must be stopped and restarted to clear all reservations; there is no reset function during runtime.	Add an admin route that loops through all 100 spots and calls `releaseSpot()` on each one without restarting the server.
The pricing multiplier is fixed at $0.30 per occupied spot and cannot be adjusted without editing and recompiling the source code.	Read pricing parameters from an external configuration file at server startup so the formula can be tuned without modifying code.
All spots are treated identically; there is no distinction between compact, handicapped, or premium spot types.	Add a `spotType` attribute to `ParkingSpot` and apply different rates depending on the category.
Any user can release any occupied spot, regardless of who originally reserved it.	Store a reservation identifier and verify it against the release request before allowing the spot to be freed.
Two users sending a reservation for the same spot at the exact same moment could produce undefined behaviour.	Add a mutex or locking mechanism around the `spots` array to prevent race conditions on simultaneous requests.
The status page requires navigating away from the main interface, breaking the single-page experience.	Fetch status data dynamically and render it on the same page using JavaScript, keeping the user on one screen.

9 GitHub Repository Structure

Repository Layout

```
Smart-Parking-Reservation-Simulator/
|-- src/
|    |-- User.cpp
|    |-- User.h
|    |-- ParkingSpot.cpp
|    |-- ParkingSpot.h
|    |-- PricingEngine.cpp
|    `-- PricingEngine.h
|-- UI-aigencode-onlyforUI/
|    |-- main.cpp
|    |-- style.css
|    `-- home.html
|-- include/
|    `-- httplib.h          % external library (~20k lines)
`-- README.md
```

Folder Structure Notes

- **src/** contains all custom class header and implementation files written by the team.
- **UI-aigencode-onlyforUI/** contains the main server file and the AI-generated frontend code (HTML, CSS). The folder name reflects that the UI code was produced through AI prompting.
- **include/** contains the `httplib.h` single-header library required for the `C++` web server. This file is approximately 20,000 lines and was not written by the team.

9.1 How to Compile and Run

Build & Run Instructions

```
# 1. Navigate to the project folder
cd Smart-Parking-Reservation-Simulator

# 2. Compile
g++ UI-aigencode-onlyforUI/main.cpp src/User.cpp
    src/ParkingSpot.cpp src/PricingEngine.cpp
    -Iinclude -Isrc -std=c++17 -o parking_server

# 3. Run the server
./parking_server

# 4. Open browser and go to localhost (port shown in terminal)

# 5. Stop the server
```

```
Ctrl + C
```

10 Tools and Technology Used

Tool / Technology	Purpose
C++	Backend language for all class logic, server routing, and data management.
HTML / CSS	Structure and styling of the web-based frontend.
JavaScript	Handles form submission via `fetch()`, sends requests to the backend, and updates the page with responses.
httplib	Single-header `C++` library that turns the program into an HTTP server.
g++	Compiler used to build the project with the `-std=c++17` flag.
Visual Studio Code	IDE used for writing, editing, and debugging code.
GitHub	Version control and team collaboration platform.

References

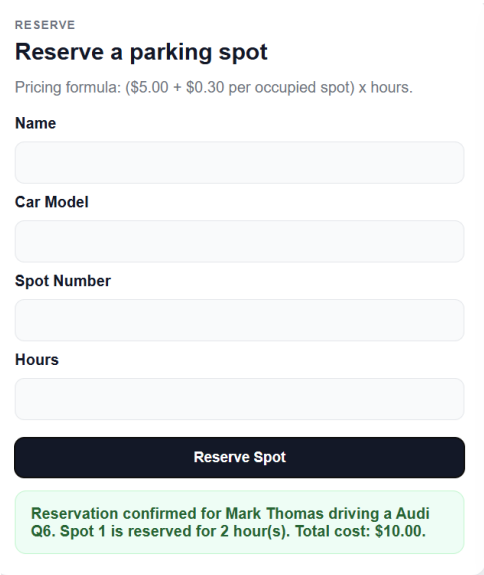
[1] Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional.

[2] yhirose. *cpp-httplib: A C++ header-only HTTP/HTTPS server and client library.* https://github.com/yhirose/cpp-httplib

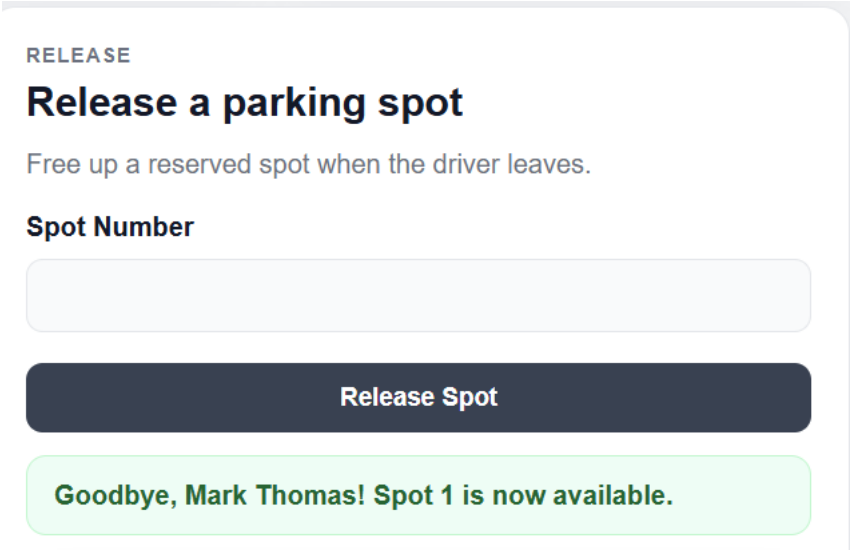
A Example Program Output

The following screenshots show the application running in a browser.

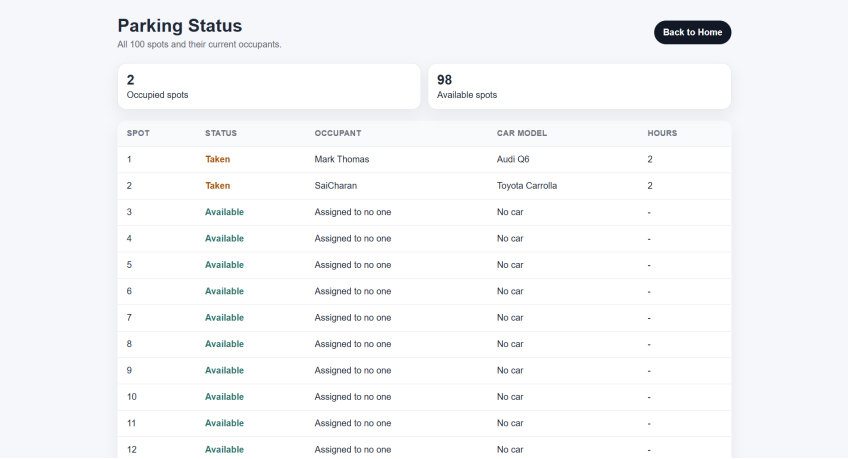
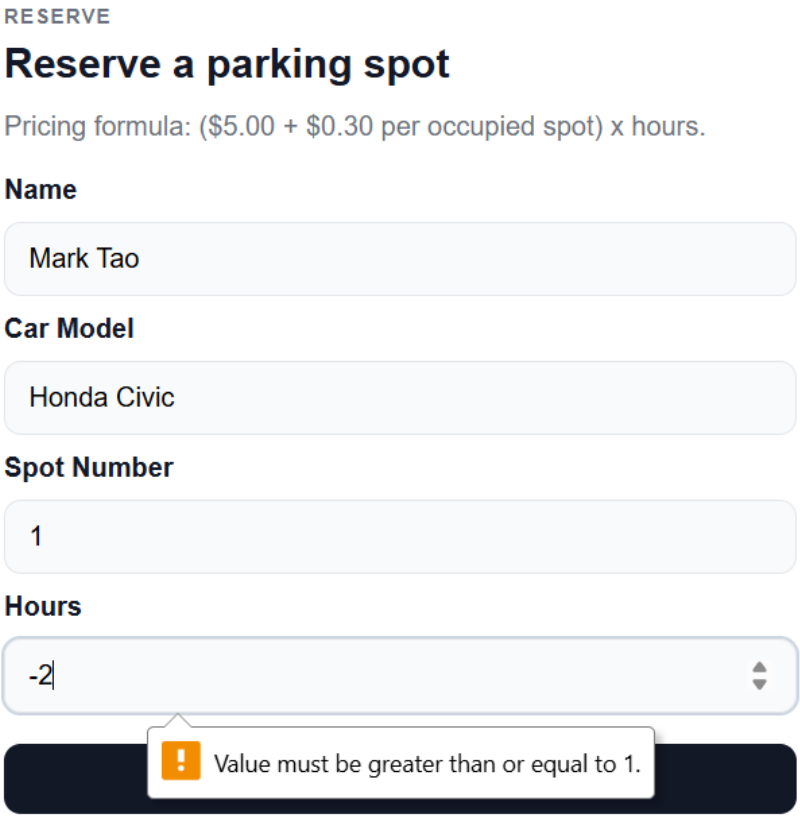
A.1 Successful Reservation



A.2 Spot Release



A.3 Status Table Example



A.4 Error Handling